DATA STRUCTURE - 5

Homework Programs

Format: Question, C Program, Sample Input/Output

Name: _____

Register No.: _____

1. Use a linked list for polynomial representation and perform addition and subtraction of polynomials.

Program (C)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int coeff, exp;
    struct Node *next;
} Node;

Node *newTerm(int c, int e) {
    Node *n = (Node *)malloc(sizeof(Node));
    n->coeff = c; n->exp = e; n->next = NULL;
    return n;
}

void addTerm(Node **head, int c, int e) {
    Node *p = *head, *prev = NULL;
    while (p && p->exp > e) { prev = p; p = p->next; }
    if (p && p->exp == e) {
        p->coeff += c;
        if (p->coeff == 0) {
            if (prev) prev->next = p->next; else *head = p->next;
            free(p);
        }
        return;
    }
    Node *n = newTerm(c, e);
    n->next = p;
    if (prev) prev->next = n; else *head = n;
}

Node *readPoly(void) {
    Node *p = NULL;
    int n, c, e, i;
    printf("Number of terms: "); scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Coefficient and exponent: ");
        scanf("%d%d", &c, &e);
        addTerm(&p, c, e);
    }
    return p;
}

Node *combine(Node *a, Node *b, int sign) {
    Node *r = NULL;
    while (a) { addTerm(&r, a->coeff, a->exp); a = a->next; }
    while (b) { addTerm(&r, sign * b->coeff, b->exp); b = b->next; }
    return r;
}

void display(Node *p) {
    if (!p) { printf("0"); return; }
    while (p) {
        printf("%dx^%d", p->coeff, p->exp);
        if (p->next && p->next->coeff > 0) printf(" + ");
        else if (p->next) printf(" ");
        p = p->next;
    }
    printf("\n");
}

int main(void) {
    Node *a, *b, *sum, *diff;
    printf("Polynomial 1\n"); a = readPoly();
    printf("Polynomial 2\n"); b = readPoly();
    sum = combine(a, b, 1);
    diff = combine(a, b, -1);
    printf("Sum: "); display(sum);
    printf("Difference: "); display(diff);
    return 0;
}
```

Sample Input / Output

```
Number of terms: 2
Coefficient and exponent: 3 2
Coefficient and exponent: 2 0
Number of terms: 2
Coefficient and exponent: 1 2
Coefficient and exponent: 4 1
Sum: 4x^2 + 4x^1 + 2x^0
Difference: 2x^2 -4x^1 + 2x^0
```

2. Find the previous node of a given node in a circular linked list.

Program (C)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int main(void) {
    Node *head = NULL, *tail = NULL, *n, *p;
    int count, i, key;
    printf("Number of nodes: "); scanf("%d", &count);
    for (i = 0; i < count; i++) {
        n = (Node *)malloc(sizeof(Node));
        printf("Data: "); scanf("%d", &n->data);
        if (!head) head = tail = n;
        else { tail->next = n; tail = n; }
    }
    if (!head) { printf("List is empty\n"); return 0; }
    tail->next = head;
    printf("Find previous node of: "); scanf("%d", &key);
    p = head;
    do {
        if (p->next->data == key) {
            printf("Previous node data: %d\n", p->data);
            return 0;
        }
        p = p->next;
    } while (p != head);
    printf("Value not found\n");
    return 0;
}
```

Sample Input / Output

```
Number of nodes: 4
Data: 10
Data: 20
Data: 30
Data: 40
Find previous node of: 10
Previous node data: 40
```

3. Insert a node at a given position in a circular linked list (positions start at 1).

Program (C)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int main(void) {
    Node *head = NULL, *tail = NULL, *n, *p;
    int count, i, value, pos;
    printf("Number of nodes: "); scanf("%d", &count);
    for (i = 0; i < count; i++) {
        n = (Node *)malloc(sizeof(Node));
        printf("Data: "); scanf("%d", &n->data);
        if (!head) head = tail = n;
        else { tail->next = n; tail = n; }
    }
    if (head) tail->next = head;
    printf("Value and position: "); scanf("%d%d", &value, &pos);
    if (pos < 1 || pos > count + 1) {
        printf("Invalid position\n"); return 0;
    }
    n = (Node *)malloc(sizeof(Node)); n->data = value;
    if (pos == 1) {
        if (!head) { head = n; n->next = n; }
        else { n->next = head; tail->next = n; head = n; }
    } else {
        p = head;
        for (i = 1; i < pos - 1; i++) p = p->next;
        n->next = p->next; p->next = n;
        if (p == tail) tail = n;
    }
    printf("Circular list: ");
    p = head;
    if (p) {
        do { printf("%d ", p->data); p = p->next; } while (p != head);
    }
    printf("\n");
    return 0;
}
```

Sample Input / Output

```
Number of nodes: 3
Data: 10
Data: 20
Data: 40
Value and position: 30 3
Circular list: 10 20 30 40
```

4. Find the second-largest element in a queue.

Program (C)

```c
#include <stdio.h>

int main(void) {
    int q[100], n, i, largest, second;
    printf("Queue size: "); scanf("%d", &n);
    if (n < 2 || n > 100) {
        printf("Enter a size from 2 to 100\n"); return 0;
    }
    for (i = 0; i < n; i++) {
        printf("Element: "); scanf("%d", &q[i]);
    }
    largest = second = -2147483647 - 1;
    for (i = 0; i < n; i++) {
        if (q[i] > largest) {
            second = largest;
            largest = q[i];
        } else if (q[i] > second && q[i] < largest) {
            second = q[i];
        }
    }
    if (second == -2147483647 - 1)
        printf("No distinct second-largest element\n");
    else
        printf("Second-largest element: %d\n", second);
    return 0;
}
```

Sample Input / Output

```
Queue size: 5
Element: 12
Element: 7
Element: 25
Element: 18
Element: 20
Second-largest element: 20
```

5. Check whether the first and third elements of a linked list are equal.

Program (C)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int main(void) {
    Node *head = NULL, *tail = NULL, *n, *p;
    int count, i;
    printf("Number of nodes: "); scanf("%d", &count);
    for (i = 0; i < count; i++) {
        n = (Node *)malloc(sizeof(Node));
        printf("Data: "); scanf("%d", &n->data);
        n->next = NULL;
        if (!head) head = tail = n;
        else { tail->next = n; tail = n; }
    }
    if (count < 3) {
        printf("List has fewer than 3 elements\n"); return 0;
    }
    p = head;
    for (i = 1; i < 3; i++) p = p->next;
    if (head->data == p->data)
        printf("First and third elements are equal\n");
    else
        printf("First and third elements are not equal\n");
    return 0;
}
```

Sample Input / Output

```
Number of nodes: 4
Data: 5
Data: 8
Data: 5
Data: 9
First and third elements are equal
```

6. Implement two stacks in one array and display both stacks.

Program (C)

```
#include <stdio.h>
#define SIZE 10

int a[SIZE], top1 = -1, top2 = SIZE;

void push1(int x) {
    if (top1 + 1 == top2) printf("Array is full\n");
    else a[++top1] = x;
}
void push2(int x) {
    if (top1 + 1 == top2) printf("Array is full\n");
    else a[--top2] = x;
}
void pop1(void) {
    if (top1 == -1) printf("Stack 1 is empty\n");
    else printf("Popped from stack 1: %d\n", a[top1--]);
}
void pop2(void) {
    if (top2 == SIZE) printf("Stack 2 is empty\n");
    else printf("Popped from stack 2: %d\n", a[top2++]);
}
void display(void) {
    int i;
    printf("Stack 1: ");
    for (i = top1; i >= 0; i--) printf("%d ", a[i]);
    printf("\nStack 2: ");
    for (i = top2; i < SIZE; i++) printf("%d ", a[i]);
    printf("\n");
}
int main(void) {
    push1(10); push1(20); push2(90); push2(80);
    display();
    pop1(); pop2();
    display();
    return 0;
}
```

Sample Input / Output

```
Stack 1: 20 10
Stack 2: 80 90
Popped from stack 1: 20
Popped from stack 2: 80
Stack 1: 10
Stack 2: 90
```

7. Flatten a linked list whose nodes have next and bottom pointers, maintaining sorted order.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next, *bottom;
} Node;

Node *make(int x) {
    Node *n = (Node *)malloc(sizeof(Node));
    n->data = x; n->next = n->bottom = NULL;
    return n;
}
Node *merge(Node *a, Node *b) {
    Node dummy, *tail = &dummy;
    dummy.bottom = NULL;
    while (a && b) {
        if (a->data <= b->data) {
            tail->bottom = a; a = a->bottom;
        } else {
            tail->bottom = b; b = b->bottom;
        }
        tail = tail->bottom;
    }
    tail->bottom = a ? a : b;
    return dummy.bottom;
}
Node *flatten(Node *root) {
    if (!root || !root->next) return root;
    root->next = flatten(root->next);
    root = merge(root, root->next);
    return root;
}
void printList(Node *p) {
    while (p) { printf("%d ", p->data); p = p->bottom; }
    printf("\n");
}
int main(void) {
    /* Three sorted columns: 5->7->8, 10->20, 19->22->30 */
    Node *a = make(5), *b = make(10), *c = make(19);
    a->bottom = make(7); a->bottom->bottom = make(8);
    b->bottom = make(20);
    c->bottom = make(22); c->bottom->bottom = make(30);
    a->next = b; b->next = c;
    printf("Flattened list: ");
    printList(flatten(a));
    return 0;
}
```

Sample Input / Output

```
Flattened list: 5 7 8 10 19 20 22 30
```

8. Merge three sorted linked lists into one sorted linked list.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *append(Node *head, int x) {
    Node *n = (Node *)malloc(sizeof(Node)), *p;
    n->data = x; n->next = NULL;
    if (!head) return n;
    p = head;
    while (p->next) p = p->next;
    p->next = n;
    return head;
}
Node *merge(Node *a, Node *b) {
    Node dummy, *tail = &dummy;
    dummy.next = NULL;
    while (a && b) {
        if (a->data <= b->data) {
            tail->next = a; a = a->next;
        } else {
            tail->next = b; b = b->next;
        }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}
void display(Node *p) {
    while (p) { printf("%d ", p->data); p = p->next; }
    printf("\n");
}
int main(void) {
    Node *a = NULL, *b = NULL, *c = NULL, *result;
    a = append(a, 1); a = append(a, 4); a = append(a, 7);
    b = append(b, 2); b = append(b, 5);
    c = append(c, 3); c = append(c, 6); c = append(c, 8);
    result = merge(merge(a, b), c);
    printf("Merged list: "); display(result);
    return 0;
}
```

Sample Input / Output

```
Merged list: 1 2 3 4 5 6 7 8
```

9. Swap the kth node from the beginning with the kth node from the end in a linked list.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int main(void) {
    Node *head = NULL, *tail = NULL, *n, *p;
    Node *x, *y, *prevX = NULL, *prevY = NULL, *prev;
    int count, i, k, temp;
    printf("Number of nodes: "); scanf("%d", &count);
    for (i = 0; i < count; i++) {
        n = (Node *)malloc(sizeof(Node));
        printf("Data: "); scanf("%d", &n->data);
        n->next = NULL;
        if (!head) head = tail = n;
        else { tail->next = n; tail = n; }
    }
    printf("Enter k: "); scanf("%d", &k);
    if (k < 1 || k > count) {
        printf("Invalid k\n"); return 0;
    }
    if (k == count - k + 1) {
        printf("Middle node; no swap needed\n"); return 0;
    }
    x = y = head; prev = NULL;
    for (i = 1; i < k; i++) {
        prevX = x; x = x->next;
    }
    for (i = 1; i < count - k + 1; i++) {
        prevY = y; y = y->next;
    }
    if (prevX) prevX->next = y; else head = y;
    if (prevY) prevY->next = x; else head = x;
    p = x->next; x->next = y->next; y->next = p;
    printf("After swapping: ");
    for (p = head; p; p = p->next) printf("%d ", p->data);
    printf("\n");
    return 0;
}
```

Sample Input / Output

```
Number of nodes: 5
Data: 1
Data: 2
Data: 3
Data: 4
Data: 5
Enter k: 2
After swapping: 1 4 3 2 5
```

10. Multiply two numbers represented by linked lists (one digit per node).

Program (C)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int digit;
    struct Node *next;
} Node;

Node *append(Node *head, int d) {
    Node *n = (Node *)malloc(sizeof(Node)), *p;
    n->digit = d; n->next = NULL;
    if (!head) return n;
    p = head;
    while (p->next) p = p->next;
    p->next = n;
    return head;
}
long long value(Node *p) {
    long long n = 0;
    while (p) { n = n * 10 + p->digit; p = p->next; }
    return n;
}
int main(void) {
    Node *a = NULL, *b = NULL;
    long long x, y, product, divisor;
    a = append(a, 1); a = append(a, 0); a = append(a, 0);
    b = append(b, 1); a = a; b = append(b, 0);
    x = value(a); y = value(b); product = x * y;
    printf("First number: %lld\nSecond number: %lld\n", x, y);
    printf("Product: %lld\nDigits in linked-list form: ", product);
    if (product == 0) printf("0");
    else {
        divisor = 1;
        while (product / divisor >= 10) divisor *= 10;
        while (divisor > 0) {
            printf("%lld ", product / divisor);
            product %= divisor;
            divisor /= 10;
        }
    }
    printf("\n");
    return 0;
}
```

Sample Input / Output

```
First number: 100
Second number: 10
Product: 1000
Digits in linked-list form: 1 0 0 0
```